

Java Backend Architecture

Interview Series

Chapter 14.2

Scalability Patterns

50+ Interview Q&A

Code Examples

Architecture Diagrams

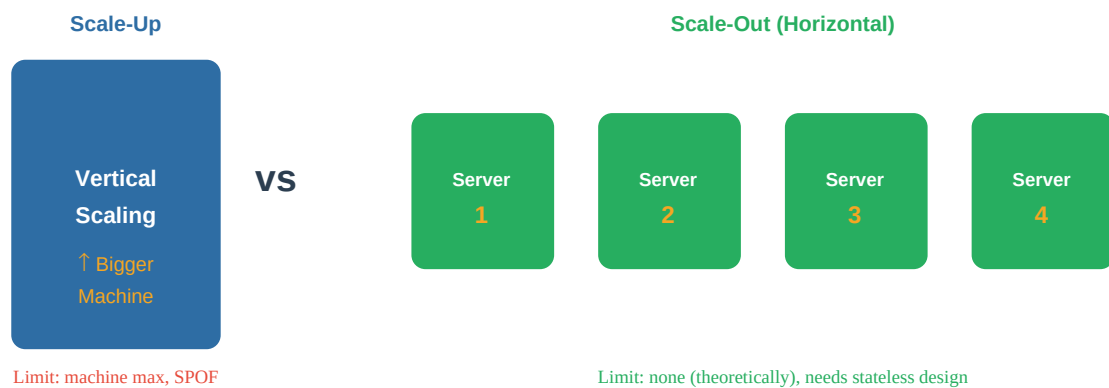
Advanced Topics

Chapter 14.2 – Scalability Patterns

Overview

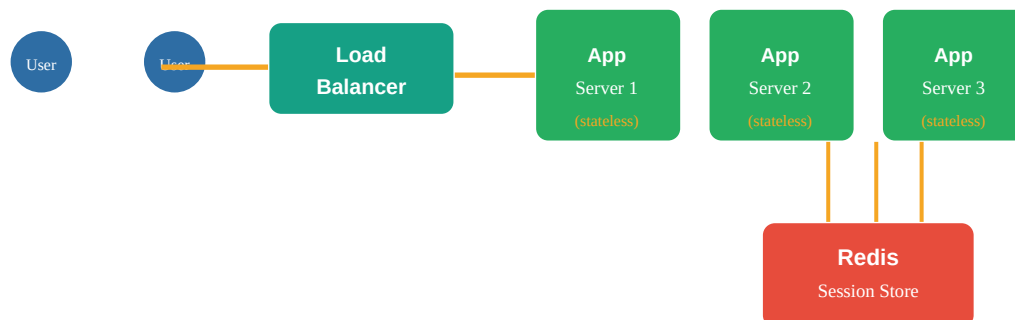
Scalability is the ability of a system to handle increasing load by adding resources. Understanding vertical vs horizontal scaling, stateless design, auto-scaling policies, and database scaling patterns is essential for senior Java backend engineers building cloud-native applications on AWS, GCP, or Azure.

Vertical vs Horizontal Scaling



Dimension	Vertical Scaling	Horizontal Scaling
Mechanism	Bigger CPU/RAM/disk on same machine	Add more identical machines
Upper Limit	Physical/cloud instance limit	Theoretically unlimited
Cost	Expensive large instances	Commodity hardware / spot instances
Failure Risk	Single point of failure (SPOF)	Redundancy built-in
Downtime	Requires restart to upgrade	Zero-downtime with rolling updates
State	Can be stateful	Must be stateless for proper distribution
Use Cases	DB primary, legacy apps	Web/API tier, microservices

Stateless Application Design



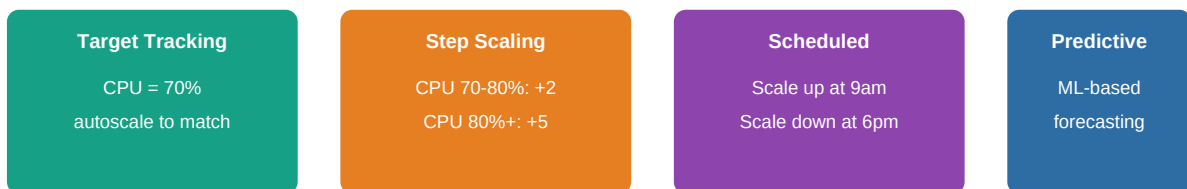
Stateless App Servers — Session Stored in Redis (not in-process)

- Stateless: server holds no session data in memory — any instance can handle any request
- Session state stored externally: Redis cluster with TTL (e.g., 30 min idle timeout)
- JWT tokens: encode user context in signed token — no server-side session lookup needed
- Benefit: add/remove instances freely, no sticky sessions, zero-downtime deploys
- Spring Boot: disable HttpSession, use Spring Session with Redis (@EnableRedisHttpSession)

```
@Configuration @EnableRedisHttpSession(maxInactiveIntervalInSeconds = 1800) public class SessionConfig { // Spring Session auto-stores HttpSession in Redis // All app server instances share session state }
```

Auto-Scaling Policies

Auto-Scaling Policy Types (AWS)



All types adjust the desired capacity of an Auto Scaling Group (ASG)

Scale-to-zero: Lambda (cold start ~100ms) | Cloud Run (cold start ~1-2s)

Target Tracking	Maintain a metric at a target value. Example: keep average CPU at 70%. ASG automatically calculates required instance count. Simplest to configure — preferred for most web workloads.
Step Scaling	Different scaling actions for different breach levels. CPU 70-80%: add 2 instances. CPU >80%: add 5 instances. More granular control. Use when traffic spikes are large and sudden.
Scheduled Scaling	Pre-defined scaling at known times. Scale out before 9am business hours, scale in at 6pm. Useful for predictable daily/weekly patterns (batch jobs, business hours traffic).
Predictive Scaling	AWS ML-based: analyzes historical patterns to forecast load, pre-scales before anticipated spike. Reduces cold start impact. Combine with target tracking for best results.
Scale-to-Zero	Lambda and Cloud Run scale to 0 instances when idle. Eliminates idle cost. Trade-off: cold start latency (Lambda ~100ms, Cloud Run ~1-2s). Not suitable for latency-sensitive paths.

Database Scaling Patterns

Pattern	Description	Use Case	Trade-off
Read Replicas	Async replication to read-only replicas	Read-heavy (reporting, search)	Replication lag, eventual consistency
Write Sharding	Split data across shards by key	Write-heavy, storage scale-out	Cross-shard queries, hot shards
Connection Pool	Reuse DB connections (HikariCP)	High concurrency	Pool exhaustion under spike

CQRS	Separate read and write models	Complex query patterns	Dual model complexity
Caching Layer	Redis/Memcached in front of DB	Repeated reads (hot data)	Cache invalidation complexity

Spring Boot Scalability Patterns

```
# application.yml – HikariCP connection pool spring: datasource: hikari: maximum-pool-size: 20
# tune to DB max_connections minimum-idle: 5 connection-timeout: 30000 # 30s max wait for
connection idle-timeout: 600000 # 10min idle before removal cache: type: redis # Spring Cache
via Redis # Read replica routing @Configuration public class DataSourceConfig { @Bean @Primary
public DataSource routingDataSource( @Qualifier("write") DataSource write, @Qualifier("read")
DataSource read) { ReadWriteRoutingDataSource ds = new ReadWriteRoutingDataSource();
ds.setTargetDataSources(Map.of("write", write, "read", read));
ds.setDefaultTargetDataSource(write); return ds; } }
```

Interview Q&A; — Scalability Patterns

Q1. What is the difference between vertical and horizontal scaling?

Vertical scaling adds more CPU/RAM/disk to a single machine — simple, no code changes, but has an upper hardware limit and creates a single point of failure. Horizontal scaling adds more machine instances — theoretically unlimited, provides redundancy, but requires stateless application design and a load balancer to distribute traffic.

Q2. Why must applications be stateless for horizontal scaling?

In a horizontally scaled system, any request can land on any server instance. If session data is stored in-process (JVM heap), the next request on a different instance won't find that session. Stateless design stores session externally (Redis, database) so any instance can serve any user's request. This enables seamless horizontal scale-out.

Q3. How does Spring Session with Redis enable stateless scaling?

`@EnableRedisHttpSession` replaces the in-process `HttpSession` with a Redis-backed session. Each session is stored in Redis with a configurable TTL. The session ID is passed in a cookie or header. All app server instances share the same Redis session store, so requests can be routed to any instance without sticky sessions.

Q4. What is the target tracking auto-scaling policy?

Target tracking maintains a CloudWatch metric at a specified target value. Example: keep average CPU utilization at 70%. AWS continuously monitors the metric and automatically adjusts instance count to maintain the target. It's the simplest policy to configure and works well for most steady-state workloads with gradual traffic changes.

Q5. What is step scaling and when should you use it?

Step scaling defines different scaling actions for different metric breach magnitudes. Example: CPU 70-80% → add 2 instances; CPU >80% → add 5 instances. Use when: traffic spikes are large and sudden (flash sales, viral events), gradual target tracking is too slow, and you know the relationship between load and required capacity.

Q6. What is predictive scaling in AWS?

AWS Predictive Scaling uses machine learning to analyze historical CloudWatch data (minimum 24h) and forecast future load. It pre-scales the Auto Scaling Group before the predicted spike arrives, eliminating warm-up lag. Best combined with target tracking as a fallback. Requires predictable daily or weekly traffic patterns to be effective.

Q7. What is scale-to-zero and what is the cold start trade-off?

Scale-to-zero means reducing instance count to 0 when there is no traffic, eliminating idle infrastructure cost. AWS Lambda and Google Cloud Run support this. Trade-off: cold start latency. Lambda cold start: ~100ms (JVM-based Spring Boot: 5-15s without SnapStart). Cloud Run: ~1-2s. Not suitable for latency-sensitive APIs; use provisioned concurrency to pre-warm.

Q8. What is Lambda SnapStart and how does it reduce cold starts?

Lambda SnapStart (Java) takes a snapshot of the JVM memory after initialization and restores it on subsequent cold starts, reducing cold start time from 5-15s to ~200ms for Spring Boot applications. Enable via `FunctionUrlConfig` or Lambda console. Requires Java 11+ runtime. Snapshot is taken during publish — microVM restores from snapshot instead of initializing JVM.

Q9. How do read replicas scale database reads?

Read replicas are asynchronous copies of the primary database. Reads (SELECT) are routed to replicas, writes (INSERT/UPDATE/DELETE) go to primary. AWS RDS supports up to 5 read replicas per primary. Use a routing DataSource in Spring to direct @Transactional(readOnly=true) transactions to replicas. Trade-off: replication lag means replicas may return slightly stale data.

Q10. What is database sharding and how does it scale writes?

Sharding horizontally partitions data across multiple database instances based on a shard key (e.g., `user_id % N`). Each shard handles writes and reads for its subset of data. Write throughput scales linearly: 4 shards = 4× write capacity. Challenges: cross-shard queries require scatter-gather, shard rebalancing is complex, and hot shards (popular users) cause imbalance.

Q11. How do you choose a shard key?

Choose a shard key with high cardinality (many unique values), uniform distribution (no hot shards), and alignment with access patterns (queries within a single shard). Common choices: `user_id` for user-centric data, `tenant_id` for multi-tenant SaaS, geographic region for location data. Avoid: timestamps (monotonically increasing causes all writes to hot shard), low-cardinality fields like status or country.

Q12. What is consistent hashing and how does it help with sharding?

Consistent hashing places both data keys and server nodes on a hash ring. Each key maps to the nearest clockwise node. When a node is added or removed, only K/N keys need remapping (K = keys, N = nodes), versus K/2 with simple modulo sharding. This minimizes cache/shard disruption during cluster topology changes.

Q13. How does HikariCP connection pooling improve scalability?

HikariCP maintains a pool of pre-established database connections, eliminating the overhead of creating a new TCP/SSL connection per query (~20-50ms). Pool size should be tuned: too small = threads wait for connections (timeout); too large = DB overwhelmed. Rule of thumb: `pool_size = cores × 2 + disk_spindle_count`. Monitor with `hikaricp.connections.acquire` metric.

Q14. What is the CQRS pattern and how does it relate to scalability?

Command Query Responsibility Segregation uses separate models for writes (commands) and reads (queries). Write model: normalized, strongly consistent. Read model: denormalized, optimized for specific query patterns, can be eventually consistent. Enables independent scaling: scale read replicas independently of write primary. Often combined with Event Sourcing.

Q15. How do you handle auto-scaling warm-up time?

New instances need time to start (JVM ~30-60s for Spring Boot, container pull ~30s, health check grace period ~60s). Total warm-up: 2-3 minutes. Mitigation: scale-out earlier (lower threshold: 50-60% CPU instead of 70-80%), maintain minimum healthy instance count, use pre-warmed Lambda with provisioned concurrency, keep Docker images small for faster pulls.

Q16. What is the difference between scale-out and scale-up in AWS?

Scale-out (horizontal): add more EC2 instances to Auto Scaling Group or more ECS tasks. Scale-up (vertical): change instance type from `t3.medium` to `m5.2xlarge`. AWS supports both. For EC2: vertical scaling requires stop/start (brief downtime). For RDS: vertical scaling requires ~5 min maintenance window. Horizontal scaling has no downtime.

Q17. What metrics should you use to trigger auto-scaling?

CPU utilization: easy to understand, good proxy for compute load (target: 60-70%). Request count per target: directly measures traffic load (ALB metric). SQS queue depth: for worker services processing queue messages. Memory utilization: not natively available, requires CloudWatch agent. Custom business metrics:

e.g., active WebSocket connections. Choose metrics that directly reflect the bottleneck.

Q18. How do you prevent auto-scaling from thrashing?

Thrashing: constant scale-out then scale-in causing instance churn. Prevention: set scale-in cooldown period (300s default) — no scale-in for 5 min after last scale-out. Scale-in conservatively: only remove 1 instance at a time. Use instance protection on recently launched instances. Target tracking has built-in scale-in dampening. Monitor for rapid oscillation in CloudWatch.

Q19. What is the N+1 scaling problem?

With N shards, adding a new shard requires rebalancing K/N keys, which causes temporary high load on the source shards. Solution: consistent hashing minimizes rebalancing. Pre-shard to more logical shards than physical nodes: e.g., 1000 logical shards on 10 nodes, each node owns 100 logical shards. When adding a node, move 100 logical shards with minimal disruption.

Q20. How does CDN offloading improve scalability?

CDN caches static assets (images, JS, CSS, videos) at edge locations globally. A well-configured CDN absorbs 70-90% of total traffic, dramatically reducing origin server load. CloudFront with S3 origin for static content is essentially infinitely scalable. Dynamic API responses with short TTLs (1-60s) can also be cached at CDN for frequently accessed, slowly changing data.

Q21. What is the bulkhead pattern in scalability?

Isolates different workload types into separate resource pools (thread pools, connection pools, service instances) so that one heavy workload cannot starve another. Example: separate thread pools for customer-facing APIs vs background batch processing. Spring: use `@Async` with custom thread pool executors. Resilience4j Bulkhead: limits concurrent calls per dependency.

Q22. How do you scale a Spring Boot application in Kubernetes?

Deploy as Kubernetes Deployment with HorizontalPodAutoscaler (HPA). HPA scales pods based on CPU/memory or custom metrics (KEDA for Kafka lag, queue depth). Configure readiness probe (/actuator/health/readiness) — pod receives traffic only when ready. Liveness probe restarts unhealthy pods. Set resource requests/limits for proper scheduling. Use PodDisruptionBudget to maintain minimum available pods during scaling.

Q23. What is the circuit breaker pattern in scalability?

Prevents cascading failures by stopping calls to a failing downstream service. States: Closed (normal), Open (failing — reject calls immediately), Half-Open (test if service recovered). Resilience4j in Spring Boot: `@CircuitBreaker` annotation, configure failure threshold (50% failures in 10s sliding window), wait duration in open state (30s), fallback method. Returns cached/default response instead of propagating failure upstream.

Q24. How do you implement rate limiting for scalability?

Rate limiting protects backend services from overload. Token bucket: allow burst up to N tokens, refill at R/sec rate. Leaky bucket: smooth out bursts to constant output rate. Redis-based: INCR + EXPIRE for sliding window counters. Spring implementation: spring-cloud-gateway RateLimiter filter uses Redis. API Gateway: AWS API Gateway throttling (method-level, account-level, per-key limits).

Q25. What is stateful vs stateless scaling and which is harder?

Stateless: each request is independent (REST API, search). Easy to scale — add instances freely. Stateful: requests depend on accumulated state (WebSocket, game sessions, workflow state). Harder to scale — must route same user to same instance (sticky sessions) or externalize state (Redis, Kafka). Externalize state whenever possible to enable horizontal scaling.

Q26. How do you handle database connection exhaustion under high load?

Symptoms: HikariCP timeout waiting for connection, DB reports too many connections. Fixes: increase pool size (but check DB max_connections limit). Use PgBouncer/RDS Proxy for connection multiplexing — many app connections → fewer actual DB connections. Reduce connection hold time (use async patterns, reduce transaction duration). Scale app servers less aggressively if DB is the bottleneck.

Q27. What is eventual consistency and how does it enable scalability?

Eventual consistency relaxes the requirement that all replicas return the latest write immediately. Replicas will converge to the same value eventually (ms to seconds). This enables: asynchronous replication (no write blocked waiting for replica ACK), geographic distribution, higher write throughput. Trade-off: reads may return stale data. Acceptable for: social feeds, product recommendations, analytics counts.

Q28. How do you scale WebSocket connections?

WebSocket is stateful (persistent connection to specific server). Scale with: NGINX as WebSocket-aware load balancer with sticky sessions (ip_hash). Pub/sub via Redis: each server subscribes to user-specific channels, routes messages to connected clients. Alternatively, use managed WebSocket service (AWS API Gateway WebSocket, Pusher, Ably). Horizontal pod autoscaler with connection count as custom metric.

Q29. What is the back-pressure mechanism?

Back-pressure prevents a fast producer from overwhelming a slow consumer. Reactive Streams standard (Project Reactor, RxJava): consumer signals demand to producer — producer only sends as many items as consumer can handle. Spring WebFlux uses Reactor which implements back-pressure natively. For queues: limit queue depth, reject/DLQ messages when full. Circuit breaker adds back-pressure at service level.

Q30. How does Spring WebFlux improve scalability vs Spring MVC?

Spring MVC: thread-per-request model (Tomcat: 200 default threads). Each blocking I/O operation holds a thread. Under high concurrency, thread pool exhausts → latency spike. Spring WebFlux: event-loop model (Netty: few threads). Non-blocking I/O — thread released during I/O wait. Handles 10× more concurrent connections with same thread count. Best for I/O-heavy APIs, not CPU-heavy computation.

Q31. What is the saga pattern for distributed scalability?

Saga manages long-lived distributed transactions without 2PC locking (which kills throughput). Each service executes a local transaction and publishes an event. Next service picks up event and executes its local transaction. On failure: compensating transactions roll back completed steps. Choreography saga (event-driven) vs Orchestration saga (central coordinator). Enables each service to scale independently.

Q32. How do you design a scalable file storage system?

Never store binary files in relational DB (BYTEA/BLOB). Use object storage: S3 (AWS), GCS, Azure Blob. Upload flow: client requests pre-signed URL from API, uploads directly to S3 (bypasses your servers), S3 notifies via event/SNS when complete, your service stores metadata (bucket, key, size, content-type) in DB. CDN in front of S3 for read-heavy (images, videos).

Q33. What is tiered storage and how does it relate to scalability?

Tiered storage matches data to cost/performance storage based on access frequency. Hot tier: SSD (NVMe) for active data accessed daily — highest performance, highest cost. Warm tier: HDD for data accessed weekly/monthly. Cold tier: S3 Standard-IA or Glacier for archival (cents/GB/month). Lifecycle policies auto-migrate data between tiers. Enables infinite-scale storage at low cost.

Q34. How do you scale a multi-tenant SaaS application?

Three tenancy models: shared everything (all tenants on same DB — cheapest, hardest to isolate), shared DB schema per tenant (one schema per tenant in same DB instance — good balance), dedicated DB per tenant (complete isolation — most expensive). For scale: shard by tenant_id, rate-limit per tenant, dedicated resources for premium tenants. Spring: AbstractRoutingDataSource for tenant-based routing.

Q35. What is the fan-out problem and how do you solve it?

Fan-out: one write event must be replicated to many destinations. Example: Twitter post must appear in all followers' timelines. Celebrity with 50M followers → 50M writes on single post. Solutions: fan-out on write (precompute, fast reads, expensive for celebrities), fan-out on read (compute at read time — always fresh), hybrid (fan-out on write for normal users, fan-out on read for celebrities with >1M followers).

Q36. How do you handle a traffic spike of 100× normal load?

Layered defense: CDN absorbs static content (immediate). Auto-scaling pre-triggered by scheduled scaling (if predictable) or step scaling (if not). Rate limiting at API gateway protects backend. Queue-based load leveling: writes go to Kafka, consumers process at sustainable rate. Graceful degradation: disable non-critical features (recommendations, analytics) to protect core flows. Circuit breakers prevent cascading failure of non-essential services.

Q37. What are the scalability limits of a single-region deployment?

Physical limits: single AZ = 1 data center SPOF. Single region: network latency >200ms for users >5000km away. Single region can handle ~1M concurrent users with proper horizontal scaling. Beyond that: multi-region active-active or active-passive. Cross-region replication for DB (RDS Global Database: <1s lag). Route53 latency-based routing directs users to nearest region.

Q38. How does event sourcing improve scalability?

Event sourcing stores state as an immutable log of events instead of current state snapshots. Benefits: append-only writes are fast and horizontally scalable. Event stream naturally supports fan-out to multiple consumers (projections). Temporal queries (state at any point). Trade-offs: read queries require replaying events (mitigated by snapshots at N events), event schema evolution requires careful versioning.

Q39. What is the two-phase deployment pattern?

For backward-compatible schema changes: Phase 1 — deploy code that handles both old and new schema (additive change: add new column with nullable or default, start writing to both). Phase 2 — after all pods run new code, drop old column/field. Never rename columns directly — add new + migrate + drop old. This enables zero-downtime schema evolution across horizontally scaled deployments.

Q40. How do you scale a search feature?

Never use SQL LIKE '%keyword%' at scale — full table scan, not index-friendly. Use dedicated search engine: Elasticsearch or OpenSearch. Index documents on write (via Logstash, Debezium CDC, or application-level dual write). Elasticsearch: distributed, horizontally scalable, inverted index for full-text search. For simple cases: PostgreSQL full-text search with GiST/GIN indexes works up to ~10M rows.

Q41. What is the difference between scale-out and partitioning?

Scale-out: add identical instances of the same service, each handling a subset of traffic. Data is replicated across instances. Horizontal scaling of the compute layer. Partitioning (sharding): divide data into non-overlapping subsets, each subset stored on a different instance. No replication overhead, each partition owns unique data. Both are used together: multiple app server instances, each connecting to the appropriate DB shard.

Q42. How do you implement graceful degradation at scale?

Feature flags (LaunchDarkly, AWS AppConfig): disable expensive features under load. Fallback responses: return cached or default data when service is slow. Timeout + fallback: Resilience4j @TimeLimiter + @Fallback. Shed load: return 503 when queue depth exceeds threshold. Prioritize core journeys (checkout > recommendations). Pre-render critical pages. Communicate status at status.yourdomain.com.

Q43. What observability tools support scalability analysis?

Metrics: Prometheus + Grafana (open source) or Datadog/New Relic (SaaS). Auto-scaling metrics in CloudWatch. Distributed tracing: Jaeger, Zipkin, AWS X-Ray — see per-service latency contribution. Profiling: Java Flight Recorder, async-profiler for CPU/memory hotspots. Load testing: k6, Gatling, Apache JMeter to validate scalability before production. Chaos engineering: inject failures with Chaos Monkey to test resilience.

Q44. What is the difference between availability and scalability?

Availability: percentage of time the system is operational and accessible (uptime). Measured as 99.9%, 99.99%. Achieved via redundancy, failover, health checks. Scalability: ability to handle growing load by adding resources without performance degradation. Achieved via horizontal scaling, sharding, caching. A system can be highly available but not scalable (single powerful server) or scalable but not available (distributed system with poor failure handling).

Q45. How do you identify scalability bottlenecks?

Load test with gradual ramp-up (k6 or JMeter): find the breaking point (latency degradation, error rate increases). Profile at breaking point: CPU-bound (add instances or optimize algorithm), memory-bound (GC pressure — heap tuning, object pooling), I/O-bound (DB query optimization, caching, connection pooling), network-bound (CDN, compression, connection reuse). Address the actual bottleneck, not the assumed one.

Q46. What is the 12-factor app methodology for scalability?

The 12-factor app: store config in environment variables (not code), keep processes stateless (process independence), treat backing services as attached resources, run dev/prod parity. Key scalability factors: VI — stateless processes (scale out by adding process instances), VII — port binding (each instance self-contained), VIII — concurrency (scale via process model). Spring Boot with externalized config (application.yml + env vars) naturally follows 12-factor.

Q47. What is Spring Cloud and how does it support scalability?

Spring Cloud provides tools for distributed systems: Eureka/Consul for service discovery (services find each other by name, not IP), Spring Cloud LoadBalancer for client-side load balancing, Spring Cloud Gateway for API gateway (routing, rate limiting), Spring Cloud Config for externalized configuration, Spring Cloud Circuit Breaker (Resilience4j integration). Together they enable building scalable, resilient microservice architectures.